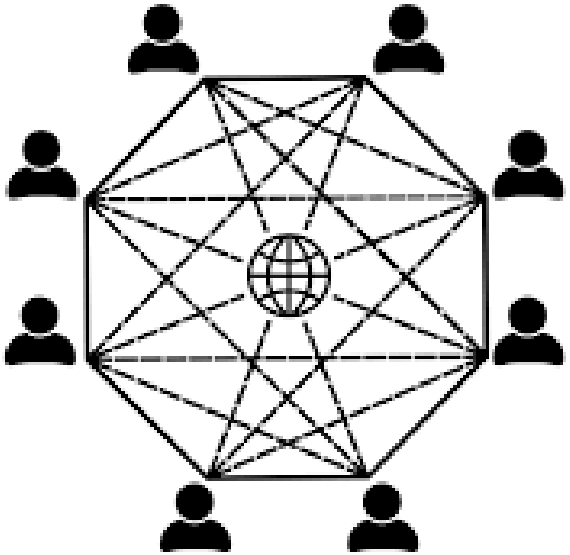
네트워크설계

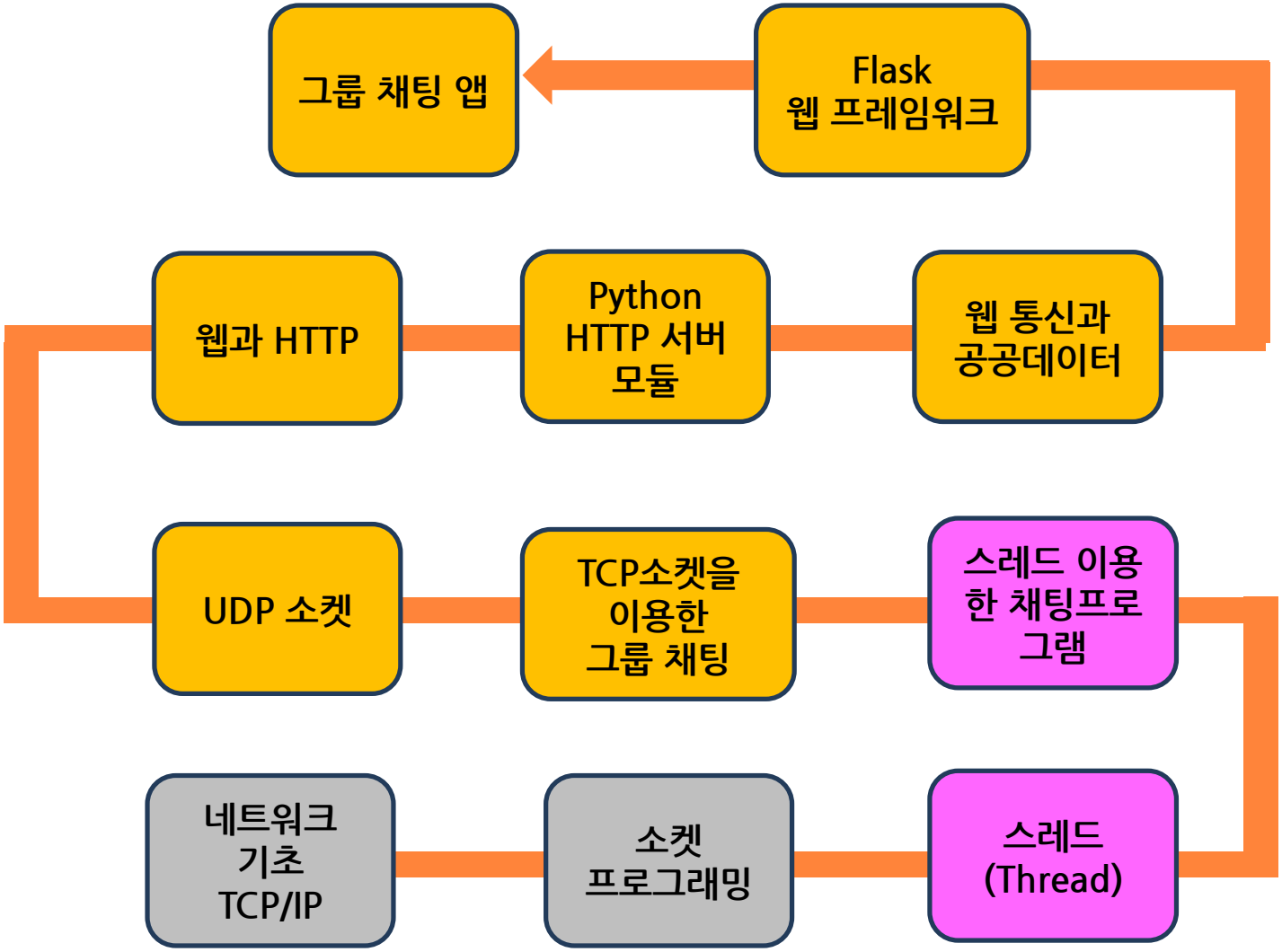
4주차 - 스레드(Thread)와 소켓 프로그래밍



인공지능학과

그림참조: https://pxhere.com/ko/photo/1637436

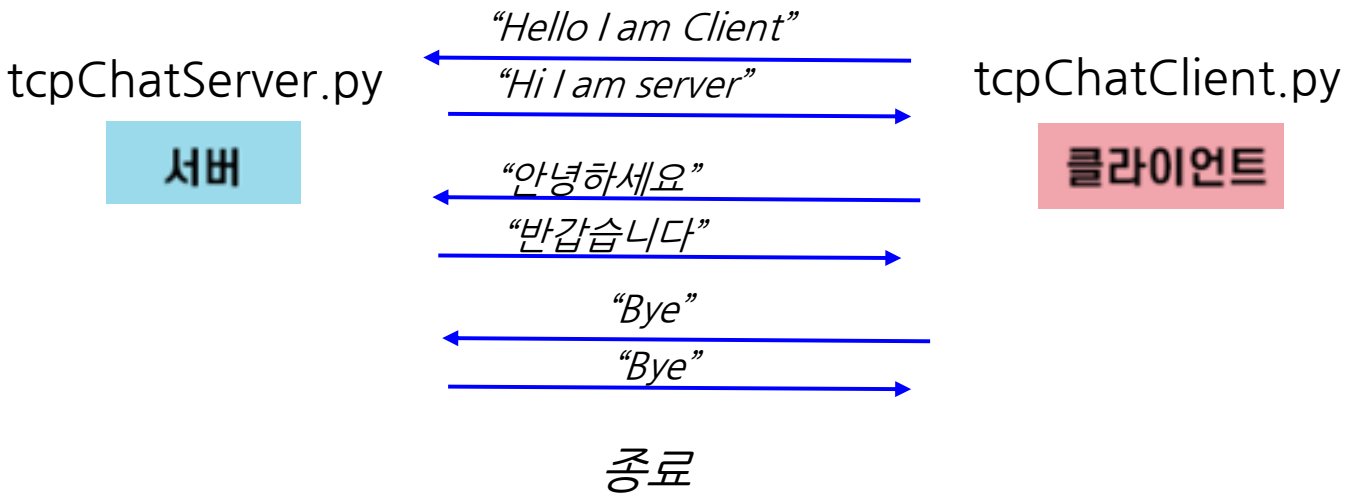
4주차는 스레드(Thread)



앱 제작 실습 병행

3주차정리. TCP 소켓 프로그램 응용 실습1 - 1:1 채팅

▪ TCP 소켓을 이용한 서버 클라이언트간 1:1 채팅 구현조건



1:1 채팅 구현 시 반영할 사항

조건1) 클라이언트가 먼저 메시지를 전송하고 서버가 응답하는 방식
조건2) 서버 클라이언트간 연결된 상태에서는 채팅은 지속할 수 있어야 한다
조건3) 서버 혹은 클라이언트가 'exit' 를 입력하면 연결은 정상 종료되어야 한다

3주차정리. TCP 소켓 프로그램 응용 실습1 - 1:1 채팅 서버

▪ TCP 소켓을 이용한 1:1 채팅 서버 프로그램

tcpChatserver.py

```python
# TCP chatting server program

import socket

IP = 'localhost'  # loopback ip (127.0.0.1)
PORT = 9999
address = (IP, PORT)  # tuple로 주소정보 저장

print(">> Server start")

# socket() 서버 소켓 생성
# AF_INET:주소종류는 IPv4, SOCK_STREAM:통신타입은 TCP
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# (optional code) winError 10048 에러 해결을 위한 추가 코드
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,1)

# bind(주소정보) 소켓에 IP주소와 포트번호 지정
server.bind(address)
# listen() 클라이언트 접속요청 대기 상태
server.listen()
print(">> Server wait.........")

# accept() 클라이언트 접속 요청이 있을 시 수락
# 통신소켓과 클라이언트 주소정보를 반환
conn, addr = server.accept()
print(f">> {addr} is connected")

while True:
    # recv(받을 메시지크기) 통신소켓에 데이터가 들어올때 까지 기다림
    # 통신소켓에 데이터가 들어오면 메시지 반환
    rxmsg = conn.recv(1024).decode()
    if not rxmsg or rxmsg == 'exit':
        print("Disconnect by Client")
        break
    print("Client >>", rxmsg)

    # sendall(보낼데이터) 통신소켓에 메시지 보내기
    sendmsg = input("Server >> ")
    conn.sendall(sendmsg.encode('utf-8'))  # 보낼때는 인코딩
    if sendmsg == 'exit':
        print("Exit by User")
        break
conn.close()      # 통신소켓 종료
server.close()       # 리스닝소켓 종료

print(">> Server End")
```

3주차정리. TCP 소켓 프로그램 응용 실습1 - 1:1 채팅 클라이언트

- TCP 소켓을 이용한 1:1 채팅 클라이언트 프로그램

tcpChatClient.py

```python
# TCP chatting client program

import socket

IP = 'localhost'  # 서버 IP
PORT = 9999       # 서버 포트번호
address = (IP, PORT)  # tuple로 주소정보 저장

print(">> Client start")

# socket() 클라이언트 소켓 생성 (ipv4, tcp 소켓)
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# connect(서버주소정보) 서버소켓으로 연결요청
client.connect(address)
print(">> Connect server")

while True:
    # sendall(보낼데이터) 서버로 메시지 보내기
    sendmsg = input("Client >>")
    client.sendall(sendmsg.encode('utf-8')) # 보낼 때 인코딩
    if sendmsg == 'exit':
        print(">> Exit by User")
        break
    # recv(받을 메시지크기) 서버 메시지 수신
    rxmsg = client.recv(1024).decode('utf-8') # 받을때는 디코딩
    if not rxmsg or rxmsg == 'exit':
        print("Disconnect by Server")
        break
    print("Sever >>", rxmsg)

# close() 클라이언트 소켓 종료
client.close()
print("Client End")
```

3주차정리. TCP 소켓 응용 실습2 - 숫자 게임

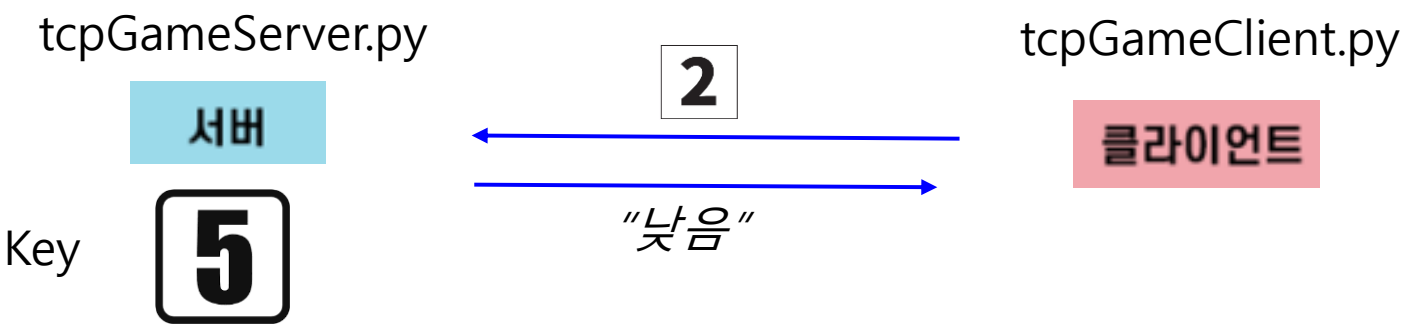
- **TCP 소켓을 이용한 서버 클라이언트간 숫자 맞추기 게임 구현하기**

작동방식: "서버는 1~9 사이의 한 숫자를 무작위로 key 값을 선정하고
클라이언트가 TCP 소켓을 이용해 접속하여 그 숫자를 맞추는 게임 "

* 서버와 클라이언트간 통신은 TCP 사용, 서버포트는 9090 사용

1) 서버는 1 ~ 9 사이의 숫자 하나를 무작위로 Key를 생성하고 클라이언트를 기다린다
2) 클라이언트는 서버에 접속하여 1~9 사이의 숫자를 전송하여 Key값을 맞추는 게임
3) 서버는 클라이언트로 받은 숫자가 Key값보다 높을 때는 "높음" ,
   낮을 때는 "낮음 " 을 전송
4) 서버가 클라이언트로 key 값을 받으면 "정답 " 이라고 응답하고
   서버와 클라이언트 연결 종료
5) 클라이언트가 중간에 '0'를 입력하면 게임 종료
6) 서버도 클라이언트로부터 '0' 을 수신하면 게임종료

주의) 숫자가 아닌 문자가 입력되면 "입력값이 잘못되었습니다" 출력하고 재입력을 기다림



3주차정리. TCP 소켓 응용 실습2 - 숫자 게임 서버

```python
# TCP Game server program w/ numbers

import socket, random

IP = 'localhost'  # loopback ip (127.0.0.1)
PORT = 9090
address = (IP, PORT)  # tuple로 주소정보 저장

print(">> Game Server start")

# socket() 서버 소켓 생성
# AF_INET:주소종류는 IPv4, SOCK_STREAM:통신타입은 TCP
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# (optional code) winError 10048 에러 해결을 위한 추가 코드
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,1)

# bind(주소정보) 소켓에 IP주소와 포트번호 지정
server.bind(address)
# listen() 클라이언트 접속요청 대기 상태
server.listen()
print(">> Game Server wait.........")

# accept() 클라이언트 접속 요청이 있을 시 수락
# 통신소켓과 클라이언트 주소정보를 반환
conn, addr = server.accept()
print(f">> {addr} is connected")

key = random.randint(1,9)  # 1~9사이 임의숫자 추출
print(f">> Server key value is {key}")

while True:
    # recv(받을 메시지크기) 통신소켓에 데이터가 들어올때 까지 기다림
    # 통신소켓에 데이터가 수신되면 메시지크기 만큼 읽기
    n = int(conn.recv(1024).decode('utf-8'))  # 받을때는 디코딩
    print(">> recv number: ", n)

    if not n or n == 0:   # gabbage 혹은 0이 수신된 경우
        print("Disconnect by Client")
        break

    # sendall(보낼데이터) 통신소켓에 메시지 보내기
    if n > key:
        conn.sendall("높음".encode('utf-8'))
    elif n < key:
        conn.sendall("낮음".encode('utf-8'))
    else:  #  수신된 숫자와 key값이 같다면
        conn.sendall("정답".encode('utf-8'))
        break

# close() 서버 소켓 종료
conn.close()      # 통신소켓 종료
server.close()    # 리스닝소켓 종료

print(">> Game Server End")
```

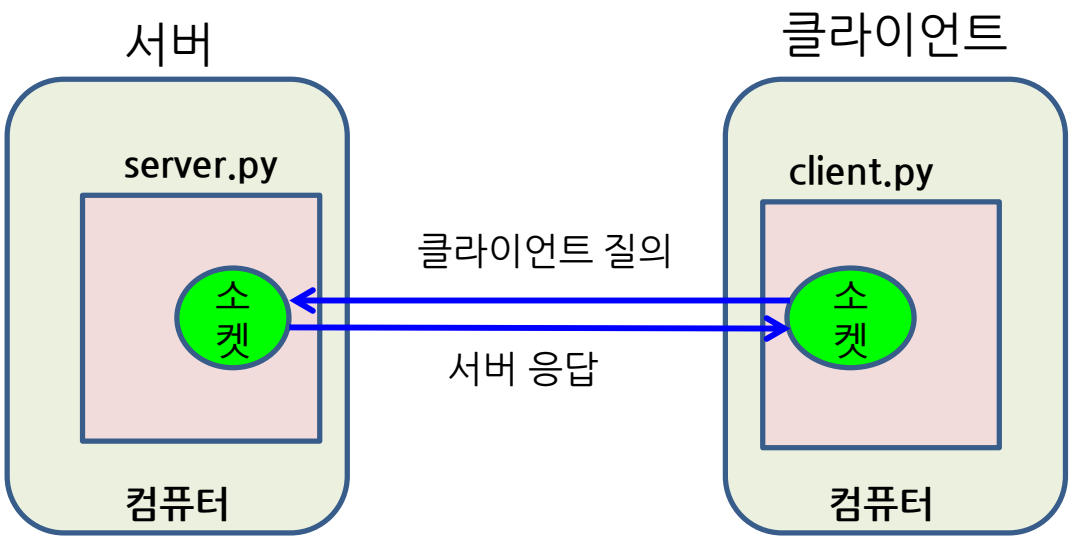
3주차정리. TCP 소켓 응용 실습2 - 숫자 게임 클라이언트

tcpGameClient.py

```python
# TCP Game client program w/ numbers

import socket

IP = 'localhost'  # 서버 IP
PORT = 9090       # 서버 포트번호
address = (IP, PORT)  # tuple로 주소정보 저장
print(">> Client start")

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(address)
print(">> Connect Game server")

while True:
    # sendall(보낼데이터) 서버로 메시지 보내기
    num = input(">> 1~9 사이의 숫자를 입력하세요(0은 게임포기): ")
    if not num.isdigit():  # 입력값이 숫자가 아닌경우
        print("입력값이 잘못되었습니다")
        continue
    client.sendall(num.encode('utf-8')) # 보낼 때 인코딩
    # recv(받을 메시지크기) 서버로부터 온 메시지 수신
    rxmsg = client.recv(1024).decode('utf-8')   # 받을때는 디코딩
    print(f"Answer >> {rxmsg}")
    if not rxmsg or rxmsg == '정답':
        break
# close() 클라이언트 소켓 종료
client.close()
print(">> Client End")
```

1. 스레드 개념과 사용법

- 3주차 TCP 1:1 채팅 프로그램 동작방식

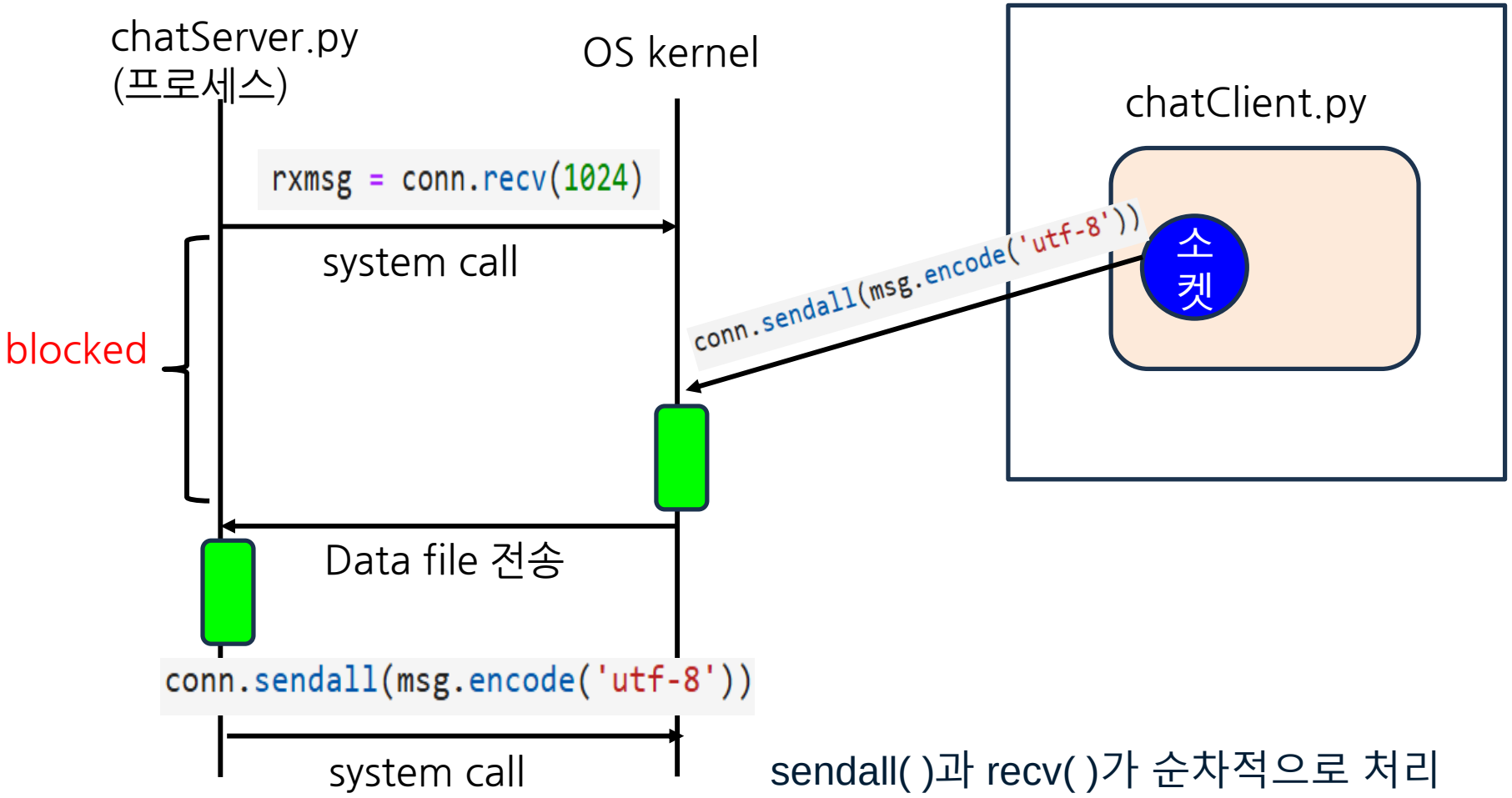


3주차에 완성한 TCP소켓을 이용한 1:1 채팅 프로그램은
클라이언트가 먼저 보내고 서버가 응답하는 제한적 방식으로 작동합니다.

순서에 상관없이 보내고 싶을 때 보내고 받는 채팅 프로그램을 제작하기 위해
무엇이 필요한지 알아봅시다”

■ 소켓 blocking I/O

- I/O를 요청한 프로세스는 요청이 완료될 때까지 블록 됨



sendall( )과 recv( )가 순차적으로 처리

1. 스레드 개념과 사용법

- 스레드(Thread) 개념

프로그램은 순차적으로 코드를 실행하기 때문에
실행 도중 소켓의 recv( )나 input( ) 함수와 같은 Blocking function을 만나면
해당 함수가 실행될 때까지 기다림

```
while True:
    rxmsg = conn.recv(1024)
    msg = rxmsg.decode('utf-8')
    print(">> recv msg: ", msg)

    msg = input(">> send msg: ")
    conn.sendall(msg.encode('utf-8'))
```

데이터 수신 전에는
메시지 송신은 안됨

데이터 입력 전에는
메시지 수신은 안됨

보내는 작업과 받는 작업을 동시에 할 수 없을까?
이 때 스레드를 이용하면 됩니다

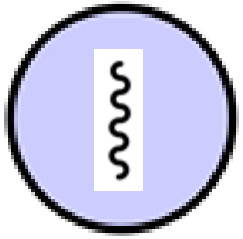
1. 스레드 개념과 사용법

■ 프로세서와 스레드

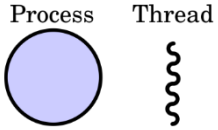
- 프로세스(Process): 실행중인 프로그램
- 스레드(Thread): 프로세스 안에서 병렬로 실행중인 하위 작업 단위
- 기본적으로 파이썬은

  프로세스는 기본적으로 하나의 싱글 스레드에서 동작 : 메인 스레드(main thread)

  메인 스레드가 코드를 순차적으로 실행



싱글 스레드
(메인 스레드)



1. 스레드 개념과 사용법

■ 스레드(Thread) 개념

- **파이썬은 싱글 스레드에서 실행(기본)**
  (실행중인 프로그램인 프로세서는 최소 1개의 main Thread로 동작)

```python
import time

def send(n):
    for i in range(n):
        print("send msg:", i)
        time.sleep(1)

def recv(n):
    for i in range(n):
        print("recv msg:", i)
        time.sleep(1)

if __name__ == "__main__":
    send(5)
    recv(5)
```

```
send msg: 0
send msg: 1
send msg: 2
send msg: 3
send msg: 4
recv msg: 0
recv msg: 1
recv msg: 2
recv msg: 3
recv msg: 4
```

send( ) 실행

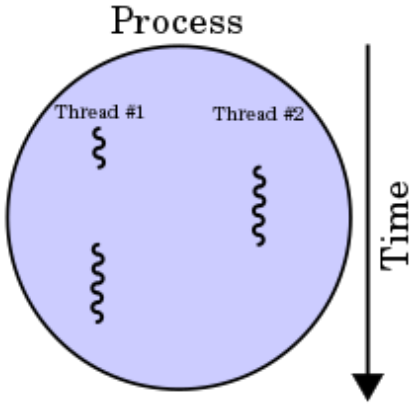
recv( ) 실행

※ 싱글 메인 스레드인 경우
send( ) 실행 종료 후 recv( ) 실행

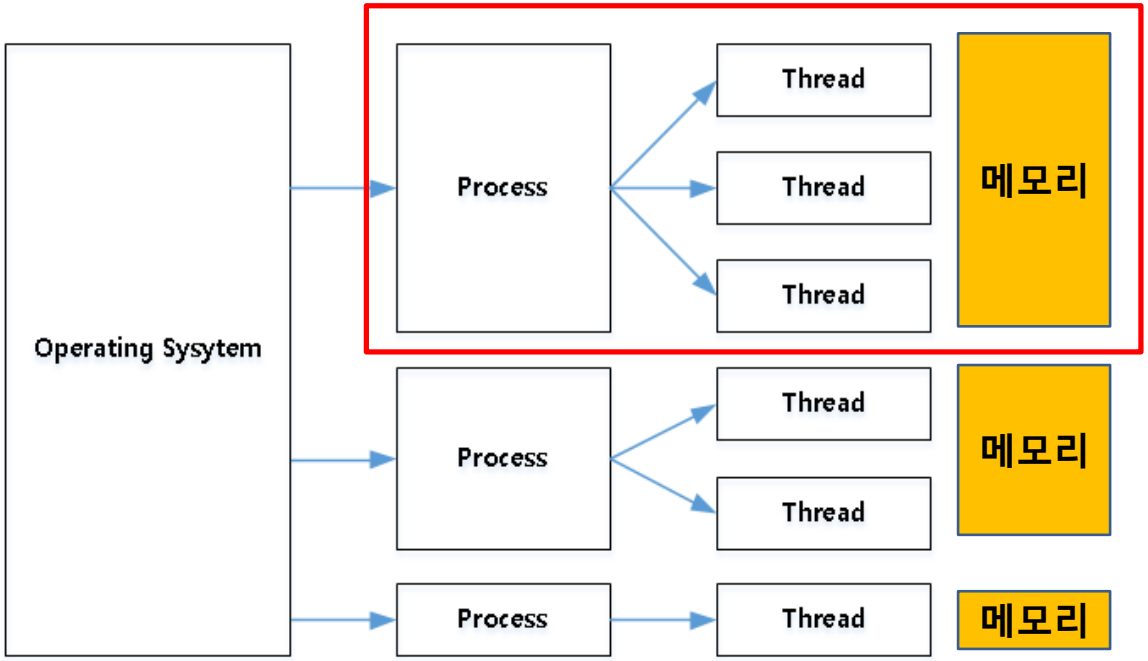
두 개의 함수를 동시에 실행할 수는 없을까?
이를 가능하게 하는 것이 멀티 스레드(Multi thread): 병렬실행

▪ 스레드(Thread)를 이용한 병렬실행

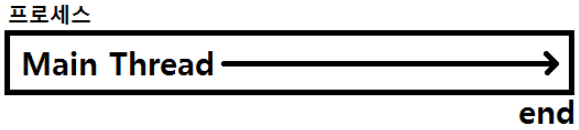
- **별도의 서브 스레드를 생성 : 멀티 스레드**
  - 멀트 스레드란 하나의 프로세스에서 둘 이상의 스레드가 동시에 작업을 수행하는 것 → 병렬 실행 (Multi Threading)
- **스레드는 자신이 속한 프로세스의 메모리를 공유**



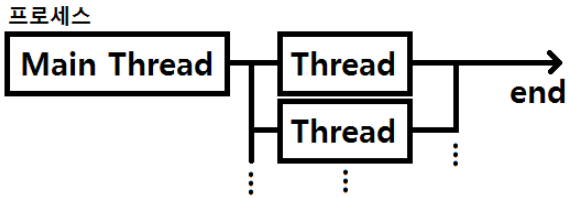
멀티 스레드



싱글 스레드



멀티 스레드



*메인 스레드: 다른 스레드를 만드는 스레드

1. 스레드 개념과 사용법 – 스레드 생성과 실행

■ 스레드 사용법

- **threading.Thread( ) 함수 호출 방식**
- threading 모듈의 threading.Thread( ) 이용하여 thread 객체 생성 후 start( ) 호출방식

```
# Thread 사용하기

import threading, time

def send(n):
    for i in range(n):
        print("send msg:", i)
        time.sleep(1)

def recv(n):
    for i in range(n):
        print("recv msg:", i)
        time.sleep(1)

if __name__ == "__main__":
    # 스레드 객체 생성하고 실행하기
    sender = threading.Thread(target=send, args=(5,))
    sender.start()
    receiver = threading.Thread(target=recv, args=(5,))
    receiver.start()
```

서브 스레드

서브 스레드

메인 스레드

```
send msg: 0
recv msg: 0
recv msg: 1
send msg: 1
recv msg: 2
send msg: 2
recv msg: 3
send msg: 3
recv msg: 4
send msg: 4
```

# 스레드 객체 생성하기 (매개변수가 있는 경우)
threading.Thread(target=함수, args=(함수의 매개변수,))
args는 튜플만 지원하며, 하나의 매개변수를 전달하는 경우, 쉼표(,)를 추가해 튜플로 설정

스레드 생성
스레드 실행

1. 스레드 개념과 사용법 – join( )

- 스레드는 언제 종료될까?

```python
# 스레드 생명주기

import threading, time

count = 0

def worker(n):
    global count
    print("worker thread start")
    for i in range(n):
        count += 1
        time.sleep(1)
    print("worker thread end")

if __name__ == "__main__":
    # 스레드 객체 생성 및 실행
    t = threading.Thread(target=worker, args=(5,))
    t.start()

    # count 출력
    print("count:", count)
```

```
worker thread start
count: 1
worker thread end
```

메인 스레드 종료

서브 스레드 종료

※ 메인 스레드가 종료되어도 서브 스레드는 자신의 작업이 끝날 때까지 계속 실행

1. 스레드 개념과 사용법 – join( )

- **메인 스레드가 스레드 종료까지 기다리려면?**
  - 서브 스레드가 종료될 때까지 기다린 후 메인 스레드를 실행하려면 스레드 join( ) 사용
  - **join( ): 다른 스레드의 종료를 기다린 후 실행**

```python
# 스레드 생명주기

import threading, time

count = 0

def worker(n):
    global count
    print("worker thread start")
    for i in range(n):
        count += 1
        time.sleep(1)
    print("worker thread end")

if __name__ == "__main__":
    # 스레드 객체 생성 및 실행
    t = threading.Thread(target=worker, args=(5,))
    t.start()
    # 스레도 종료시까지 대기
    t.join()

    # count 출력
    print("count:", count)
```

```
worker thread start
worker thread end
count: 5
```

서브 스레드 작업 종료 후
메인 스레드 실행

1. 스레드 개념과 사용법 – 이벤트 객체 사용

- 무한루프 스레드는 언제 종료될까?

* 스레드가 한 번 만들어지면 강제 종료시킬 수 있는 방법이 없음

```python
# 무한루프 스레드 종료하기

import threading, time

def loop():
    while True:
        time.sleep(1)
        print("Infinite Loop thread!")

if __name__ == "__main__":
    # 스레드 객체생성 후 실행하기
    t = threading.Thread(target=loop)
    t.start()

    print("main thread start")
    time.sleep(5)
    print("main thread end")
```

```
main thread start
Infinite Loop thread!
Infinite Loop thread!
Infinite Loop thread!
Infinite Loop thread!
main thread end
Infinite Loop thread!
Infinite Loop thread!
Infinite Loop thread!
Infinite Loop thread!
```

메인 스레드 작업 종료

서브 스레드 계속 실행

메인 스레드가 종료될 때 서브 스레드를 종료시키는 방법은?

- 스레드를 데몬 스레드(daemon thread)로 동작시키는 방법 (t.daemon = True)
- 스레드 이벤트 객체를 사용하는 방법

1. 스레드 개념과 사용법 – Event(이벤트) 객체 사용

스레드 실습3: 이벤트 객체를 사용해 서브 스레드 종료 시키기

- threading 모듈의 Event 객체를 사용
- Event 객체의 set( )을 호출하면 객체의 내부 플래그 값을 True로 설정
- is_set( )을 사용하여 플래그 값을 확인 후 서브 스레드를 종료

```python
# 무한루프 스레드 종료하기 - 이벤트 객체 사용

import threading, time

#이벤트 객체 생성
event = threading.Event()

def loop():
    while True:
        time.sleep(1)
        if event.is_set(): # 이벤트 객체 flag 확인
            print("Infinite Loop thread stop!")
            return
        print("Infinite Loop thread!")

if __name__ == "__main__":
    # 스레드 객체생성 후 실행하기
    t = threading.Thread(target=loop)
    t.start()

    print("main thread start")
    time.sleep(5)
    event.set()   # 이벤트 객체 flag set
    print("main thread end")
```

```
main thread start
Infinite Loop thread!
Infinite Loop thread!
Infinite Loop thread!
Infinite Loop thread!
main thread end
Infinite Loop thread stop!
```

메인 스레드 종료 시 서브 스레드 종료

2. 스레드간 통신 – 큐 사용

- 1~ n까지 더하는 total_sum( ) 함수를 스레드로 동작시켜 봅시다

```
# 스레드간 통신하기

import threading

def total_sum(start, end):
    result = 0
    for i in range(start, end):
        result += i
    print("result:", result)

if __name__ == "__main__":
    start = 0
    end = 1000000
    # 스레드 객체 생성 및 실행
    t = threading.Thread(target=total_sum, args=(start, end+1))
    t.start()
```

서브 스레드

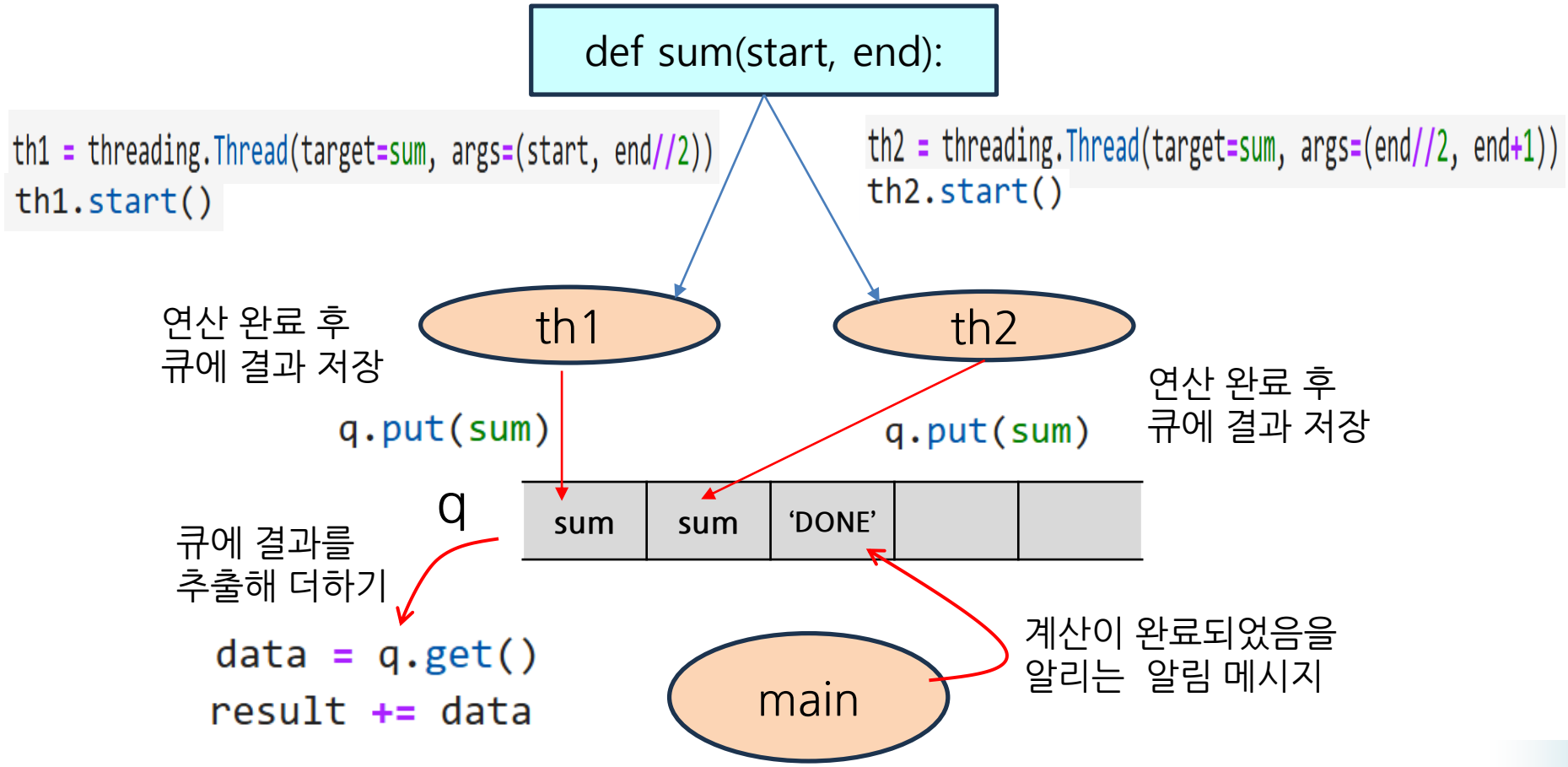
메인 스레드

```
result: 500000500000
```

2. 스레드간 통신 – 큐 사용

- **스레드간 통신에 큐 사용하기 : 두 스레드로 나누어 연산하기**
  - 프로세스 내에서 스레드 간 통신을 구현할 때 큐를 사용 → 메시지 큐
  - 큐에 데이터 삽입은 **put(data)** , 추출은 **get( )**

“ 0 ~ 1,000,000 더하기를 두 스레드로 나누어 연산하기”



2. 스레드간 통신 – 큐 사용

■ 스레드간 통신에 큐 사용하기

```python
# 스레드간 통신하기 - 큐 사용
import threading, queue

def sum(start, end, q):
    sum = 0
    for i in range(start, end):
        sum += i
    q.put(sum)
    return

if __name__ == "__main__":
    q = queue.Queue()  # 큐 생성

    start = 0
    end = 1000000
    # 스레드 객체 생성 및 실행
    t1 = threading.Thread(target=sum, args=(start, end//2, q))
    t2 = threading.Thread(target=sum, args=(end//2, end+1, q))
    t1.start()
    t2.start()
```

큐 모듈 추가

q

sum	sum	'DONE'		

계산 완료 후 큐에 결과 저장

```python
    t1.join()
    t2.join()

    q.put("Done")
    result = 0

    while True:
        data = q.get()
        if data == "Done":
            break
        else:  # 두 스레드 결과 더하기
            result += data
    print("total sum:", result)
```

스레드 종료
기다렸다 종료되면
큐에 'Done' 저장

큐 내용
순서대로 추출
해서 더하기

```
total sum: 500000500000
```

2. 스레드간 통신 - 응용실습

- **과제: 토끼와 거북이의 경주에서 누가 이겼는지 스레드 사용해서 코드로 구현하기**

경주 길이는 총 30m
- 거북이는 1초에 1m의 속도,
- 토끼는 0.5초에 1m의 속도로 달린다.
  토끼는 25m 지점에서 낮잠을 자고 17초 후에 깨어난다
  낮잠에서 깨어난 토끼는 나머지 구간을 0.3초에 1m 속도로 달린다
- 경주가 완료된 후 최종 승자를 출력한다

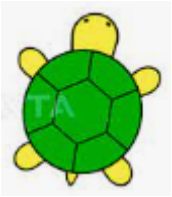
위 조건인 경우 최종 승자는 누구인지 코드로 확인하기
단, 달린 거리는 1m 단위로 출력 (예: 거북이-1m, 토끼-8m)

그림출처: rajathathablog. the rabbit vs the tortoise

2. 스레드간 통신 - 응용실습

■ 참조 코드

- 거북이 기본코드 : 1초에 1m의 속도로 총 30m를 달린다



```
# 거북이는 1초에 1m씩 총 30m
def turtle():
    for i in range(1, 31):
        time.sleep(1)
        print(f"거북이-{i}m")
    print(f"거북이-30m 도착")
    return
```

+ 스레드간 통신기능(큐) 추가

- 토끼 기본코드 완성하기 (거북이 코드 참조)

def rabbit() :

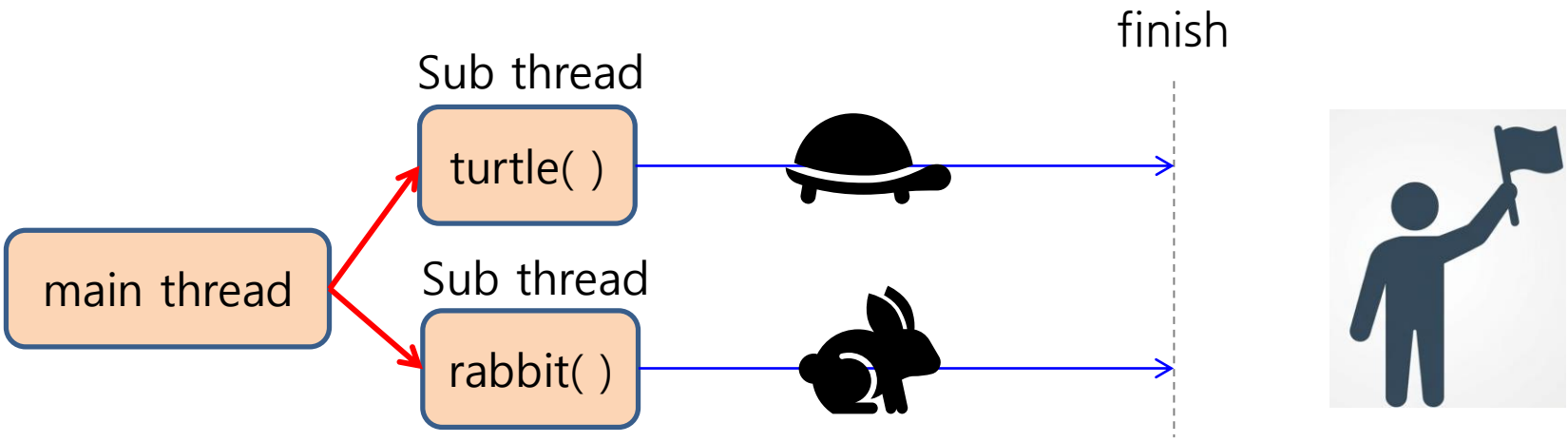
- 0.5초에 1m의 속도로 달린다
- 토끼는 25m 지점에서 낮잠을 자고 17초 후에 깨어난다 (time.sleep( ) 활용)
- 낮잠에서 깨어난 토끼는 나머지 구간을 0.3초에 1m 속도로 달린다

+ 스레드간 통신기능(큐) 추가

과제 해결 순서

과제 아이디어1:
turtle( )과 rabbit( ) 함수를 스레드로 만들어 병렬처리

과제 아이디어2:
main 스레드는 두 스레드 종료까지 기다렸다가 경주 결과를 출력하기



2. 스레드간 통신 - 응용실습

- 다음과 같이 출력이 되도록 프로그래밍 하세요

거북이 스레드
- 1초에 1m의 속도로 총 30m

토끼 스레드
- 0.5초에 1m의 속도로 달린다
- 토끼는 25m 지점에서 낮잠을 자고 17초 후에 깨어난다 (time.sleep( ) 활용)
- 낮잠에서 깨어난 토끼는 나머지 구간을 0.3초에 1m 속도로 달린다

```
경주 시작
토끼   -0m
거북이-0m
토끼   -1m
토끼   -2m
거북이-1m
토끼   -3m
  ⋮
토끼   -25m
토끼 25m 낮잠----------
  ⋮
토끼 25m 잠깸----------
 토끼 -26m
  ⋮
 ?  -30m
     도착
    -29m
    -30m
    도착
토끼와 거북이 경주는 ? 가 이겼습니다
```

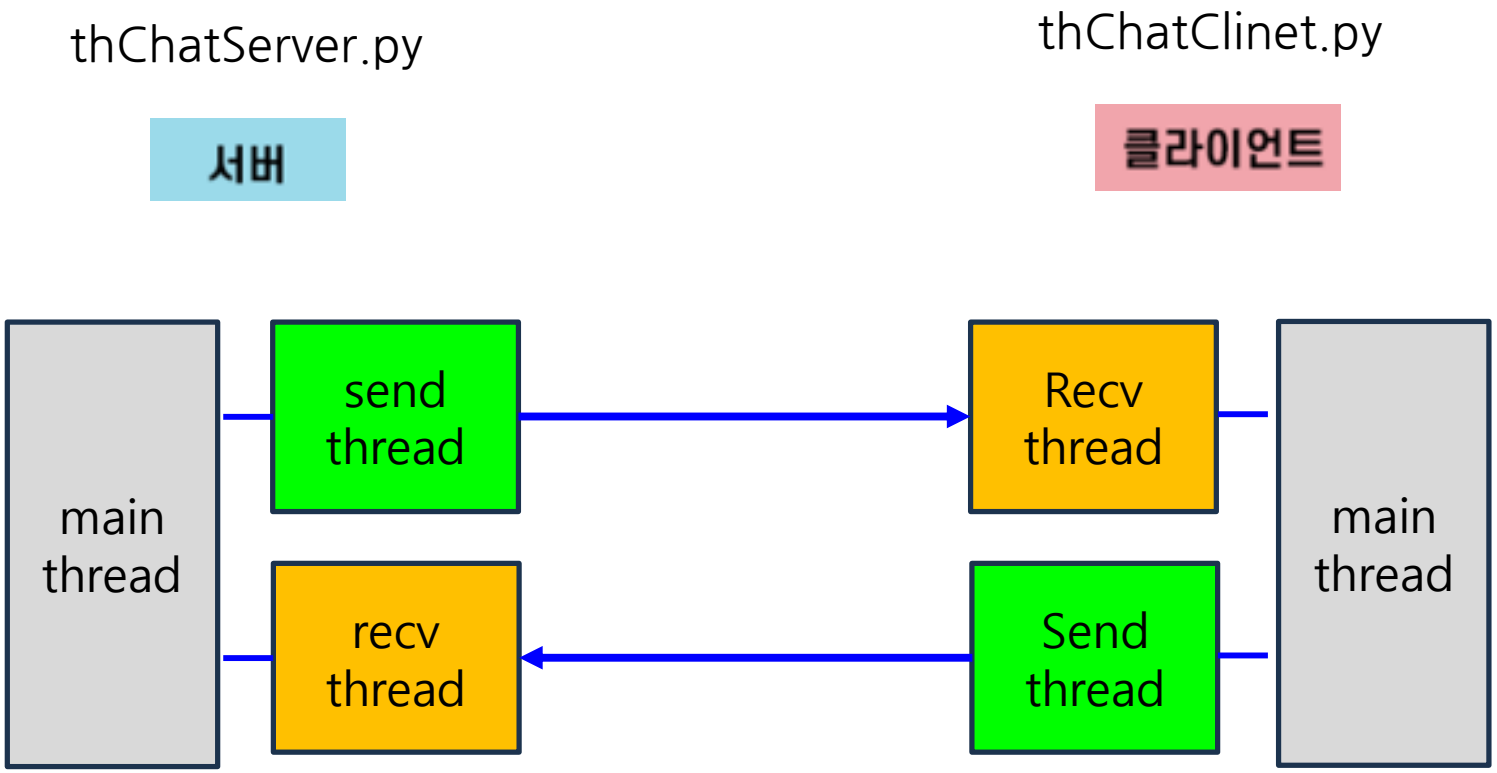
토끼와 거북이 스레드: 1m마다 출력

토끼 스레드: 낮잠 출력

토끼 스레드: 잠깸 출력

스레드와 TCP 소켓을 이용해 1:1 채팅하기

- 스레드를 이용해 언제든 메시지를 주고 받을 수 있도록 채팅 프로그램을 구현해 봅시다



3. 스레드와 TCP 소켓을 이용한 1:1 채팅 프로그래밍

- TCP 소켓을 이용한 1:1 채팅 서버 프로그램을 스레드로 실행시키기

```python
# accept() 클라이언트 접속 요청이 있을 시
# 통신소켓과 클라이언트 주소정보를 반환
conn, addr = server.accept()
print(f">> {addr} is connected")

while True:
    # recv(받을 메시지크기) 통신소켓에 데이터가 들어올때 까지 기다림
    # 통신소켓에 데이터가 들어오면 메시지 반환
    rxmsg = conn.recv(1024).decode()
    if not rxmsg or rxmsg == 'exit':
        print("Disconnect by Client")
        break
    print("Client >>", rxmsg)

    # sendall(보낼데이터) 통신소켓에 메시지 보내기
    sendmsg = input("Server >> ")
    conn.sendall(sendmsg.encode('utf-8'))  # 보낼때는 인코딩
    if sendmsg == 'exit':
        print("Exit by User")
        break
conn.close()        # 통신소켓 종료
server.close()        # 리스닝소켓 종료
```

```python
receiver = threading.Thread(target=Recv,args=(conn,))
receiver.start()
sender = threading.Thread(target=Send,args=(conn,))
sender.start()
```

```python
def recv(socket):
    while True:
        rxmsg = conn.recv(1024).decode()
        if not rxmsg or rxmsg == 'exit':
            print("Disconnect by Client, please 'exit'...")
            break
        print("Client >>", rxmsg)
```

```python
def send(socket):
    while True:
        sendmsg = input("Server >> ")
        conn.sendall(sendmsg.encode('utf-8'))
        if sendmsg == 'exit':
            print("Exit by User")
            break
```

3. 스레드와 TCP 소켓을 이용한 1:1 채팅 프로그래밍 - 서버

- 스레드와 TCP 소켓을 이용한 1:1 채팅 서버 프로그램 (파일: th_tcpChatServer.py)

```python
# TCP chatting server program w/ thread

import socket, threading

def recv(socket):
    while True:
        rxmsg = socket.recv(1024).decode()
        if not rxmsg or rxmsg == 'exit':
            print("Disconnect by Client, please 'exit'...")
            break
        print("Client >>", rxmsg)

def send(socket):
    while True:
        sendmsg = input("Server >> ")
        socket.sendall(sendmsg.encode('utf-8'))
        if sendmsg == 'exit':
            print("Exit by User")
            break

if __name__ == "__main__":
    IP = 'localhost'  # loopback ip (127.0.0.1)
    PORT = 9999
    address = (IP, PORT)  # tuple로 주소정보 저장

    print(">> Server start")
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # (optional code) winError 10048 에러 해결을 위한 추가 코드
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,1)
    server.bind(address)
    server.listen()
    print(">> Server wait.........")
    conn, addr = server.accept()
    print(f">> {addr} is connected")
    # 메시지 송신과 수신을 스레드로 생성하고 실행
    receiver = threading.Thread(target=recv, args=(conn,))
    receiver.start()
    sender = threading.Thread(target=send, args=(conn,))
    sender.start()
    while True:
        receiver.join()
        sender.join()
        break
    conn.close()      # 통신소켓 종료
    server.close()       # 리스닝소켓 종료
    print(">> Server End")
```

3. 스레드와 TCP 소켓을 이용한 1:1 채팅 프로그래밍 - 클라이언트

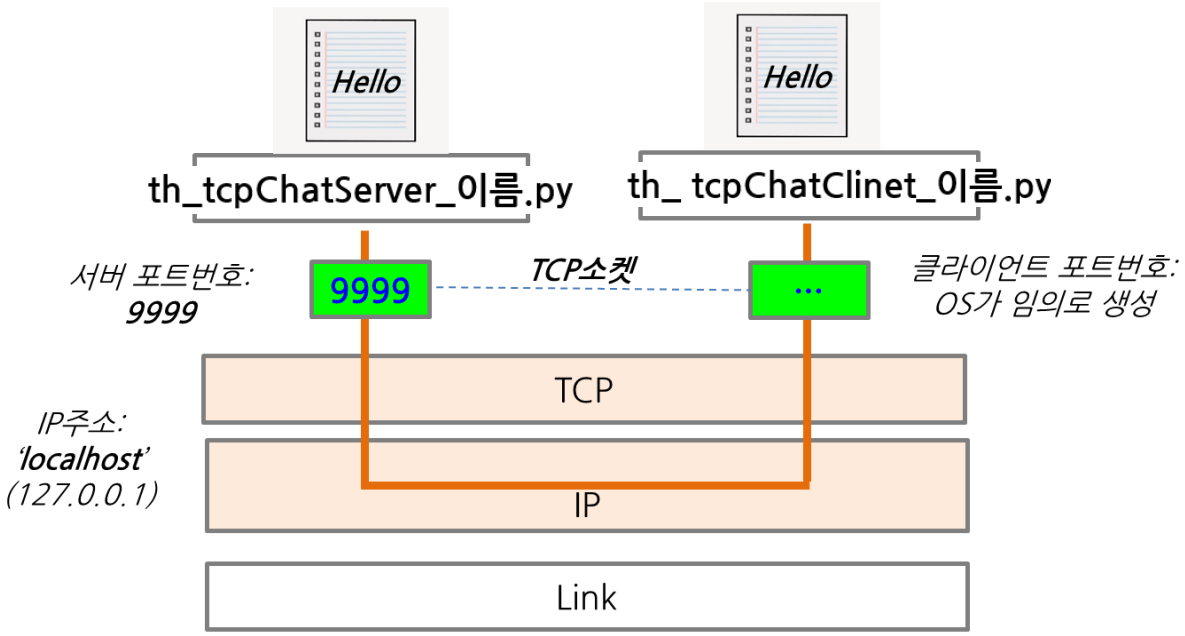
- 스레드와 TCP 소켓을 이용한 1:1 채팅 클라이언트 프로그램(파일: th_tcpChatClinet.py)

```python
# TCP chatting client program w/ thread

import socket, threading

def send(socket):
    while True:
        sendmsg = input("Client >>")
        socket.sendall(sendmsg.encode('utf-8'))
        if sendmsg == 'exit':
            print(">> Exit by User")
            break

def recv(socket):
    while True:
        rxmsg = socket.recv(1024).decode('utf-8')
        if not rxmsg or rxmsg == 'exit':
            print("Disconnect by Server, please 'exit'...")
            break
        print("Sever >>", rxmsg)

if __name__ == "__main__":
    IP = 'localhost'  # 서버 IP
    PORT = 9999       # 서버 포트번호
    address = (IP, PORT)  # tuple로 주소정보 저장

    print(">> Client start")
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect(address)
    print(">> Connect server")
    # 메시지 송신과 수신을 스레드로 생성하고 실행
    receiver = threading.Thread(target=recv, args=(client,))
    receiver.start()
    sender = threading.Thread(target=send, args=(client,))
    sender.start()

    while True:
        receiver.join()
        sender.join()
        break

    client.close()
    print("Client End")
```

4주차 실습과제 제출안내

과제명: 스레드를 이용한 채팅 프로그램

- 제출방법: 파이썬 파일을 PC에 저장 후 과제 제출란에 제출
- 첨부파일: **th_tcpChatServer_이름.py , th_ tcpChatClinet_이름.py**
- 프로그램 상 오류 혹은 이해가 안 되는 부분은 comment 남겨 주세요.



One more step: 서로 다른 컴퓨터와 채팅하기

■ 네트워크를 통해 서로 다른 컴퓨터와 채팅하기

- 이제껏 localhost(127.0.0.1) looping 주소를 가지고 내컴퓨터 내에서 실습을 진행했습니다.
- 실습 환경을 바꾸어 실제 네트워크 상에서 다른 컴퓨터와 채팅 메시지를 주고 받아봅시다

- **옆 사람(다른 컴퓨터)와 짝을 지어 한 사람은 서버, 다른 사람은 클라이언트가 되어 채팅 메시지를 주고 받아 봅시다** (항상 서버가 먼저 동작되어야 함)

- **확인할 사항 (스레드의 효과를 확인해 봅시다)**

1) 서버와 클라이언트가 정상 연결이 되는지

2) 서버와 클라이언트가 채팅 메시지를 언제든 주고 받을 수 있는지

Thank You